

Независимый от поставщика контракт ModelClient

Основной план, модуль 01

Краткое содержание

`ModelClient` отделяет код приложения от протокола конкретного поставщика и выполняет ровно одно обращение к LLM. `ModelRequest` описывает намерение приложения, `ModelResponse` — завершённый результат, а `ModelEvent` — одно типизированное событие потокового ответа. Общий контракт должен нормализовать семантику, необходимую вызывающему коду, но одновременно сохранять исходные блоки, идентификаторы, порядок, причины остановки и сведения об использовании. Потеря неизвестного поля опаснее его временного непонимания.

Ключевые понятия

Понятие	Зачем нужно	Чего не делает
<code>ModelClient</code>	Асинхронная граница одного обращения: выбирает адаптер, преобразует запрос, нормализует ответ и ошибки	Не поддерживает цель <code>Agent</code> и не выполняет <code>tool</code>
<code>ModelRequest</code>	Независимо от поставщика выражает сообщения, доступные <code>tool</code> , требуемый формат результата и нужные возможности	Не копирует структуру HTTP-запроса одного API
<code>ModelResponse</code>	Хранит упорядоченные результаты одного завершённого обращения: текст, структурированные данные, <code>tool call</code> , отказ, сведения об использовании	Не является только строкой и не скрывает причину завершения
<code>ModelEvent</code>	Представляет один смысловой шаг потока: начало, часть текста, часть аргументов, завершение блока, сведения об использовании или завершение ответа	Не обещает, что частичный JSON уже можно исполнять
<code>provider data</code>	Исходные блоки поставщика и неизвестные расширения, сохранённые без потерь рядом с общей формой	Не должны попадать в бизнес-логику без адаптера

Основные идеи

Проблема: одинаковая семантика, разные формы

OpenAI Responses возвращает упорядоченные `output Items`, Anthropic Messages — `content blocks` и `stop_reason`, Gemini `generateContent` — `candidates` с `content.parts`, `finishReason` и `usageMetadata`. Во всех трёх системах может появиться текст, структурированный результат, `tool call`, отказ или ограничение длины. Но совпадение смысла не означает совпадения данных.

Если приложение читает каждую форму напрямую, протокол поставщика проникает в Agent loop, тесты и бизнес-правила. Если адаптер оставляет только строку, он теряет `call_id`, `safety-данные`, `citations`, `reasoning Items`, `thoughtSignature`, подробности кэша и новые типы блоков. Нужны одновременно общая форма и `lossless escape hatch` — доступ к исходным данным без переупаковки с потерями.

Один поток выполнения

```
application
-> ModelRequest
-> ModelClient
-> provider adapter -> provider API
<- raw response или raw stream events
<- ModelResponse или ModelEvent stream + сохранённые provider data
```

Для обычного вызова адаптер разбирает весь ответ и возвращает `ModelResponse`. Для потока он выдаёт `ModelEvent` в исходном порядке; сборка всех событий должна приводить к тому же смысловому `ModelResponse`, что и непотоковый вызов. Если поток оборвался, частичный результат остаётся наблюдаемым, но не становится успешным завершённым ответом.

Что приводить к общей форме

Общая семантика	Нормализованное представление
Текст	Упорядоченные текстовые блоки, а не только склеенная строка
Структурированный ответ	Исходный JSON-текст, проверенное значение и идентификатор применённой схемы
<code>tool call</code>	Идентификатор вызова, имя, аргументы и порядок; выполнение остаётся снаружи <code>ModelClient</code>
Сведения об использовании	Общие <code>input</code> , <code>output</code> , <code>total</code> , если поставщик их сообщил; отсутствие остаётся отсутствием
Результат генерации	<code>completed</code> , <code>tool_requested</code> , <code>refused</code> , <code>truncated</code> , <code>blocked</code> или <code>incomplete</code>
Поток	Начало, части блоков, завершение блоков, <code>usage</code> и терминальное событие
Ошибка границы	Общая категория с исходной причиной, статусом, <code>request ID</code> и <code>retry_after</code> , если они есть

Строка, похожая на JSON, не становится структурированным ответом сама по себе. Для этого `ModelRequest` должен потребовать схему, а адаптер — проверить результат по этой схеме.

Что сохранять без потерь

- provider name, фактические model/version и response/request IDs;
- исходные блоки в исходном порядке вместе с неизвестными вариантами;
- provider-specific stop reason, status и детали отказа;
- citations, safety ratings и prompt feedback;
- reasoning Items, encrypted content, opaque fingerprints и caller linkage;
- Gemini `thoughtSignature` и порядок частей, необходимый для продолжения;
- детальные usage-поля: `cached`, `cache write`, `reasoning`, `server-tool usage`;
- HTTP status, заголовки `request-id` / `retry-after` и исходное тело ошибки;
- типы и данные неизвестных потоковых событий.

«Без потерь» означает сохранить полученные байты либо структурное дерево без отбрасывания полей. Преобразовать неизвестный блок в `dict`, удалить поля и затем сериализовать обратно — уже не lossless round trip.

Примеры

Примеры сокращены до полей, важных для сравнения; это не полные ответы API.

1. OpenAI Responses: `function_call`

```
{
  "id": "resp_oai_1",
  "status": "completed",
  "output": [{
    "type": "function_call", "call_id": "call_1",
    "name": "get_weather", "arguments": "{\"city\":\"Казань\"}"
  }],
  "usage": {"input_tokens": 42, "output_tokens": 18}
}
```

Общая форма получает `tool call` с `call_id`, именем и проверяемыми аргументами. Нельзя оставить только SDK helper `output_text`: Responses использует массив типизированных Items, где рядом могут находиться `message`, `reasoning` и другие блоки. Исходный Item сохраняется целиком.

2. Anthropic Messages: отказ как результат

```
{
  "id": "msg_ant_1", "type": "message",
  "content": [{"type": "text", "text": "Не могу выполнить запрос."}],
  "stop_reason": "refusal",
  "stop_details": {"type": "refusal", "category": "policy"},
  "usage": {"input_tokens": 31, "output_tokens": 9}
}
```

Это успешный HTTP-обмен с терминальным результатом `refused`, а не `ModelTransportError`. Общая форма сохраняет видимый текст, но `stop_reason` и `stop_details` нужны для политики fallback и аудита. В потоке эти сведения появляются позднее текста в `message_delta`.

3. Gemini `generateContent`: структурированный текст и метаданные

```
{
  "candidates": [{
    "content": {"role": "model", "parts": [
      {"text": "{\"umbrella\":true}", "thoughtSignature": "opaque..."}
    ]},
    "finishReason": "STOP", "safetyRatings": []
  ]},
  "usageMetadata": {"promptTokenCount": 24, "candidatesTokenCount": 7},
  "modelVersion": "provider-version", "responseId": "gem_1"
}
```

После проверки запрошенной схемы JSON можно представить как структурированный результат. `thoughtSignature`, `safetyRatings`, `modelVersion`, `responseId` и исходный `Part` нельзя выбрасывать. Если `promptFeedback.blockReason` присутствует и `candidates` отсутствуют, это нормальный заблокированный результат, а не ошибка разбора массива.

Потоковые различия

- OpenAI Responses выдаёт typed SSE events, например `response.created`, `response.output_text.delta`, `response.function_call_arguments.delta` и `response.completed`.
- Anthropic Messages выдаёт `message_start`, жизненный цикл каждого content block, `message_delta` и `message_stop`; `input_json_delta.partial_json` нельзя разбирать как завершённый JSON до `content_block_stop`.
- Gemini `streamGenerateContent` возвращает последовательность `GenerateContentResponse`; новый Interactions API использует события `step.start`, `step.delta`, `step.stop` и `interaction.completed`.

`ModelEvent` должен выражать общую семантику, но всегда нести исходный тип и данные события. Неизвестный тип передаётся как `unknown provider event`, а не молча игнорируется.

Заметки и наблюдения

Классификация ошибок

Где возникло	Общая категория	Повтор и данные, которые нельзя потерять
Требуемая возможность отсутствует до запроса	UnsupportedCapability	Не повторять на том же маршруте; сохранить требование
Неверный API key или недостаточно прав	ModelAuthenticationError	Не повторять без изменения конфигурации; status/request ID
Некорректный запрос	ModelRequestRejected	Не повторять без исправления; provider error code и body
Rate limit или quota	ModelRateLimited	Только ограниченный повтор; retry_after , quota scope, request ID
Перегрузка или 5xx	ModelUnavailable	Ограниченный повтор с budget; status и provider error type
Истёк локальный deadline	ModelTimeout	Повтор — новое обращение и новая стоимость; фаза запроса
Сбой сети	ModelTransportError	Сохранить исходное исключение и факт получения байтов
Неизвестная/невалидная форма ответа	ModelProtocolError	Обычно не повторять тем же адаптером; raw body/event
Поток оборван после частей ответа	ModelStreamInterrupted	Сохранить все ModelEvent и признак неоднозначной полноты
Внешняя отмена	CancelledError	Передать без обёртывания и не запускать скрытый повтор

refused , truncated , blocked , tool_requested и остановка по stop sequence — терминальные результаты LLM, а не транспортные исключения. Так вызывающий код может отдельно решать, показать отказ, продолжить после tool , увеличить лимит или выбрать другую LLM.

Матрица контрактных тестов

Один набор поведения запускается для каждого адаптера и для детерминированной имитации. Названия provider-полей различаются, утверждения — нет.

Контракт	OpenAI	Anthropic	Gemini	Детерминированная имитация
Текст и порядок блоков	<code>output Items</code>	<code>content []</code>	<code>parts []</code>	Тот же <code>ModelResponse</code>
Структурированный ответ проверен схемой	<code>text format</code>	<code>output format</code>	<code>response schema</code>	Валидный и невалидный JSON
<code>tool call</code> сохраняет ID/name/args	<code>function _call</code>	<code>tool _use</code>	<code>functionCall</code>	Скриптовый вызов
<code>tool</code> никогда не исполняется в <code>ModelClient</code>	✓	✓	✓	Счётчик равен нулю
Usage нормализован, детали сохранены	<code>usage</code>	<code>usage</code>	<code>usageMetadata</code>	Есть и отсутствует
Отказ — результат, не transport error	<code>refusal block/status</code>	<code>refusal</code>	<code>safety/prompt feedback</code>	Скриптовый отказ
Truncation не маскируется как success	<code>incomplete details</code>	<code>max_tokens</code>	<code>MAX_TOKENS</code>	Скриптовый предел
Сборка событий равна обычному ответу	<code>typed events</code>	<code>block lifecycle</code>	<code>response chunks</code>	Один сценарий, две формы
Частичный JSON не исполняется	<code>arguments delta</code>	<code>input JSON delta</code>	<code>Interactions arguments delta; generateContent</code> отдаёт завершённый call	Оборванная последовательность
Неизвестный block/event сохранён	✓	✓	✓	Генерируемый unknown
Auth/rate/timeout/protocol mapped	✓	✓	✓	По одному отказу каждого типа
<code>CancelledError</code> проходит без замены	✓	✓	✓	Детерминированная отмена

Контрактные тесты проверяют наблюдаемое поведение, а не классы SDK. Иначе «общий» набор фактически закрепит реализацию первого адаптера.

Типичные ошибки

1. Свести ответ к `text: str` и потерять параллельные блоки, IDs и citations.

2. Считать любой JSON-подобный текст структурированным ответом без запроса и проверки схемы.
3. Исполнять `tool call` в `ModelClient`, смешивая протокол и полномочия.
4. Разбирать аргументы `tool` до терминального события потока.
5. Превращать отсутствие `usage` в нули и тем самым искажать стоимость.
6. Считать `refusal`, `safety block` или `token limit` сетевой ошибкой.
7. Падать на новом `provider block` либо молча его отбрасывать.
8. Повторять внутри адаптера без общего `retry budget` и наблюдаемой причины.

Ограничения этой фазы

Это контракт уровня семантики, а не Python-интерфейсы. Здесь нет SDK, настоящих API-вызовов, реализации сборщика потока и обещания единого наименьшего подмножества возможностей. Поля, которых нет у части поставщиков, должны оставаться optional, а не получать выдуманные значения. Реализация и исполняемые контрактные тесты начнутся только после команды `к практике`.

Вопросы для повторения

1. Почему `ModelResponse(text=...)` недостаточен даже для ответа, который визуально содержит только текст?
2. Где должен исполняться `tool call` и какую информацию обязан передать `ModelClient`?
3. Почему отсутствие `usage` нельзя нормализовать в нулевые токены?
4. Что должен сделать адаптер с новым неизвестным `content block`?
5. Как доказать тестом эквивалентность потокового и обычного ответа?

Итоги

Общий контракт должен стабилизировать смысл, а не копировать JSON первого поставщика и не сводить все ответы к строке. `ModelRequest` выражает намерение, `ModelClient` владеет одним обменом, `ModelEvent` сохраняет причинный порядок потока, а `ModelResponse` представляет завершённый результат. Нормализованные поля делают Agent loop независимым от поставщика; сохранённые `provider data` предотвращают потерю новых возможностей и диагностической информации.

Источники

Проверено 2026-07-31:

- OpenAI Responses: mapping messages to typed Items
- OpenAI Responses: typed streaming events
- OpenAI API specification, `POST /v1/responses` — OpenAPI 2.3.0
- Anthropic Messages API: stop reasons
- Anthropic Messages API: streaming lifecycle
- Anthropic API versioning — `anthropic-version: 2023-06-01`; новые optional fields, enum values и event types могут добавляться

- Gemini `GenerateContentResponse`
- Gemini API versions — Interactions API доступен в stable `v1` ; SDK по умолчанию использует `v1beta`
- Gemini migration to Interactions API — `generateContent` считается legacy, но остаётся поддерживаемым
- Основной план: промпт 1.2